Sistemas Expertos

Integrantes del grupo:
Alexandra Castaño Celis
Sebastian Naranjo Bernal

Presentado a:
Valentina Tabares Morales

Asignatura:
Sistemas Inteligentes Computacionales

Fecha de entrega:
21/10/2026

Universidad Nacional de Colombia
Sede Manizales

Tabla de Contenido

I. Arquitectura..... 2
II. Representación de hechos..... 3
III. Estrategia de conflictos y decisiones..... 4
IV. Uso de herramientas de inteligencia artificial generativa..... 4

I. Arquitectura

El proyecto consiste en desarrollar un sistema experto basado en reglas que permitan identificar posibles causas de problemas comunes de conectividad de red y proporcionar recomendaciones para solucionarlos.

El sistema recibe hechos iniciales que describen el estado de una conexión, los compara con una base de conocimiento y aplica reglas para deducir posibles causas. Finalmente, presenta al usuario los pasos de inferencia y las recomendaciones obtenidas.

La arquitectura se divide en varios componentes en donde cada uno de estos, cumple con una responsabilidad especifica:

- **Conocimiento/red.json:** Contiene la base de conocimiento, compuesta por 17 reglas relacionadas con problemas de conexión, cableado, Wi-Fi, DHCP, DNS y configuración del equipo.
- **ejemplos/:** En esta parte están los archivos JSON con casos de prueba y sus hechos iniciales, como los casos de DNS, DHCP y resolución de conflictos.
- **src/cargador.py:** Lee los archivos JSON, valida elementos básicos de su estructura y prepara las reglas y los hechos para utilizarlos en el programa.
- **src/conflictos.py:** Define los criterios - estrategias utilizados para seleccionar una regla cuando existen varias reglas aplicables.
- **src/motor.py:** Implementa el motor de inferencia, que compara los hechos conocidos con los antecedentes de las reglas, aplica las reglas seleccionadas y genera nuevos hechos.
- **src/app.py:** Implementa la interfaz gráfica mediante Streamlit. Además de esto, permite cargar archivos, seleccionar hechos, ejecutar la inferencia paso a paso y consultar los resultados.
- **tests/test_motor.py:** Contiene pruebas automatizadas para verificar el funcionamiento del motor de inferencia.
- **base_codi_clase/motor_originalProf.py:** Aquí se encuentra el código original del motor de inferencia el cual fue proporcionado por la profe como referencia para desarrollar el proyecto.

II. Representación de hechos

Para almacenar la información utilizamos el formato JSON, que permite representar datos estructurados mediante objetos, listas y valores. Se eligió porque es legible, fácil de modificar y compatible con Python.

Los hechos representan condiciones conocidas del problema. Por ejemplo, en un diagnóstico de conectividad pueden utilizarse los siguientes hechos:

- ping_gateway_ok: el equipo logra comunicarse con el router.
- ping_ip_publica_ok: el equipo logra comunicarse con una dirección IP pública.
- ping_dominio_falla: existe un problema al acceder mediante un nombre de dominio.
- ip_169_254: el equipo presenta una dirección IP del rango que Windows puede asignar automáticamente cuando no obtiene una dirección mediante DHCP.

Los hechos iniciales se almacenan en archivos externos. Un ejemplo de su representación es:

```
{"hechos": {"ping_gateway_ok": true, "ping_ip_publica_ok": true,
"ping_dominio_falla": true, "otros_dispositivos_ok": true}}
```

Para este caso los 4 hechos se consideran verdaderos y pueden utilizarse para activar reglas relacionadas con los problemas DNS.

Las reglas siguen una estructura basada en antecedentes y consecuentes. Por ejemplo:

```
{"nombre": "R06 Falla DNS", "antecedentes": ["ping_ip_publica_ok", "ping_dominio_falla"],
 "consecuentes": ["falla_dns"],
 "prioridad": 6,
 "explicacion": "Hay salida por IP pero no se resuelven nombres."}
```

Los antecedentes son las condiciones que deben cumplirse para que se aplique una regla. Los consecuentes son los nuevos hechos que se deducen al aplicarla. La prioridad se utiliza como criterio de selección cuando hay varias reglas candidatas, y la explicación permite informar al usuario por qué se aplicó una regla.

De esta manera, el sistema puede obtener nuevos hechos a partir de los iniciales y utilizarlos en reglas posteriores.

III. Estrategia de conflictos y decisiones

El motor utiliza una estrategia de **encadenamiento hacia adelante.** Este método parte de los hechos iniciales y busca las reglas cuyos antecedentes se cumplen con los hechos conocidos.

Cuando encuentra reglas aplicables, utiliza la estrategia de resolución de conflictos para seleccionar cuál ejecutar. Después, agrega los consecuentes de la regla seleccionada a la memoria de trabajo y vuelve a buscar reglas aplicables. El proceso continúa hasta que no quedan reglas que puedan ejecutarse.

La estrategia de resolución de conflictos contempla los siguientes criterios:

Para definir nuestra estrategia de resolución de conflictos, tomamos como referencia los criterios de Rolston y Bigus, autores presentados por la profesora en clase. A partir de sus propuestas, seleccionamos y adaptamos algunos criterios según las necesidades del proyecto, incorporando también la prioridad de las reglas y un desempate determinista para obtener resultados consistentes.

1. **No repetición de reglas -** Bigus (criterio 3) y Rolston (criterio 1): el motor guarda las reglas que ya se ejecutaron para evitar que vuelvan a dispararse durante la misma sesión.
2. **Prioridad** - criterio propio del proyecto: se selecciona la regla con el valor de prioridad más alto. Este criterio se relaciona con la saliencia utilizada en CLIPS.
3. **Especificidad** - Bigus (criterio 1): si varias reglas tienen la misma prioridad, se favorece la que tenga mayor número de antecedentes.
4. **Recencia** - Bigus (criterio 2): si continúa el empate, se favorece la regla que tenga más antecedentes entre los hechos agregados por la última regla ejecutada. Este criterio toma como referencia el principio de recencia de OPS5.
5. **Desempate por orden de definición** - adaptación propia del criterio aleatorio de Bigus (criterio 4): si las reglas siguen empatadas, se selecciona la primera según su posición en el archivo de conocimiento, en lugar de elegir una al azar. Esto permite obtener resultados consistentes y reproducibles.

IV. Uso de herramientas de inteligencia artificial generativa

Durante el desarrollo del proyecto utilizamos ChatGPT y Claude como apoyo para entender mejor algunos conceptos de los sistemas expertos, aclarar dudas sobre el funcionamiento del sistema y organizar la documentación. También nos ayudaron con algunas partes del código y con la revisión de ciertos componentes.

Además, utilizamos estas herramientas para revisar ideas sobre la estructura del proyecto y mejorar las explicaciones de su funcionamiento. Las respuestas y sugerencias de estas herramientas nos sirvieron como guía durante el desarrollo, pero nosotros nos encargamos de revisar el código, hacer las pruebas y comprobar que el sistema funcionara correctamente.